第四届开放原子大赛 · 2026 开源行业解决方案创新赛

赛道一 · 开源基础软件与解决方案

RAGAudit
检索评测审计工具

不给你的 RAG 又一个分数，而是审计这个分数能不能信

项目名称	RAGAudit：RAG 检索评测审计工具
申报主体	学生团队
开源协议	Apache-2.0
代码仓库	https://github.com/MaPleooo9/RAGAudit
镜像仓库	https://gitee.com/iKnowuHateme/RAGAudit
文档版本	参赛提交版 · 2026 年 10 月

本文件为参赛作品介绍文档，所有数据均来自仓库内可复现产物。

一、项目背景与问题定义

1.1 一个数字不构成证据

检索增强生成（RAG）是目前大模型落地最主流的工程形态之一，而检索质量是它的命门： 召回的文档不对，后面无论换多强的生成模型都救不回来。因此几乎每个 RAG 项目都会对外报一个检索指标， 例如「命中率 94.1%」。

但这行数字本身不构成任何证据。同一个 94.1%，可能来自扎实的语义检索， 也可能来自四种系统性虚高——而它们**全部发生在算分环节，不在回答环节**， 因此现有的答案质量评测框架看不见：

虚高来源	具体表现	现有工具能否发现
判定口径过松	标题里包含期望实体名就算命中，实际指向的是另一个页面	否。只报一个口径，无从对照
题库信息泄漏	问句里直接写着答案实体名，实体匹配即可命中	否。默认题库是可信的
规则抢功	分数其实来自别名表、实体硬置顶等规则，而非语义检索	否。需人工注释代码再跑一遍
删题涨分	悄悄移除难例，平均分自然上升	否。只比较总分，不比对题目集合

这四类问题有一个共同点：**它们都是「评测方法」的缺陷，而不是「系统能力」的缺陷。** 用答案质量评测去查检索分数是否可信，方向就是错的。

1.2 项目定位

> RAGAudit 不产出「又一个分数」，它回答的是：这个分数怎么来的、虚高了多少、 是哪个部件挣来的、下一次改动弄坏了哪几条。

换句话说，它把「评测」本身当成被审计对象。与主流框架是**分层关系而非替代关系**：

	RAGAudit	RAGAS	DeepEval
量什么	**分数本身是否可信**	答案是否有据、是否切题	整条链路与 Agent 行为
判定方式	确定性硬指标，**不调 LLM**	LLM-as-judge	LLM-as-judge（含 G-Eval）
可复现性	相同输入 → 完全相同输出	换 judge 就换一把尺子	同左
运行时依赖	**零**（纯标准库）	ragas 及数据集生态	deepeval

推荐分工：RAGAudit 守「检索不退化」（秒级、可进 CI），RAGAS / DeepEval 守「答案不退化」。

1.3 目标用户

- **RAG 应用开发者**：需要在上线前知道自己的指标有没有水分，在迭代中知道哪一条退化。
- **评测集维护者**：需要检查题库是否存在实体泄漏、难例是否被悄悄删掉。
- **技术评审与验收方**：需要在不复跑整套系统的前提下判断对方报的分数可不可信。

二、核心能力（对应评审维度「技术创新」）

2.1 双口径：把「虚高」变成一个可计算的差值

同一份检索结果，按两套规则各算一遍：

- `lenient`：结果带 `target` 字段就比 target；没有则看标题是否**包含**期望名。
- `strict`：优先比 target；没有则要求页面主实体**完全相等**。

两个数字的差值就是虚高量。报告永远并列呈现，不挑好看的那个报。 差 10 个百分点不是误差，而是有 10% 的命中共依赖宽松匹配。

> **一个真实的设计教训。**早期版本按层名分派口径（给 `l1` 配包含匹配）。 当使用者换了一套层命名（`guide` / `faq` / `ref`）， lenient 会**静默退化成 strict**——指标数字还在，含义已经全变了，而人眼看不出来。 现版本把分派依据改到「结果自身有没有 `target`」这一客观字段上，与层命名彻底解耦。

2.2 消融与归因：这个分数是谁挣的

把某个部件关掉再跑一遍，`ΔHit@1` 就是它的净贡献。关键在于：**关掉之后反而变好的部件会被单独标出。**负收益模块不该以「还没证明有害」为理由留在链路上——这正是真实案例中发现的第一个可删模块。

```
VARIANTS = [
    ("full（含别名）",      search_v1, "线上方案"),
    ("full -alias（关俗称）", search_v2, "验证别名表的净贡献"),
]
```

```
ragaudit ablate -s suite.json -v variants.py --layers l2,l3
```

2.3 难度画像：拆穿「自己出题自己考」

不需要人工分组，直接从题面判定每道题的信息泄漏等级，共四档：

等级	含义	意味着什么
`explicit`	问句直接写着答案实体名	实体匹配就能命中，测不出检索能力
`alias`	问句用的是已登记的俗称	测的是别名表覆盖度，不是语义
`partial`	问句含同名子串，易指向错误实体	比完全不含实体名更难，也更容易判错期望值
`implicit`	整句不含任何实体名	**只有这一档才真正考验语义检索**

显式指名占比超过 70% 时，`lint` 会直接给出警告：这个总分大概率是查词典查出来的。 这把「题库质量」从主观判断变成了一个可核对的数字。

2.4 逐条回归：坏在哪一环

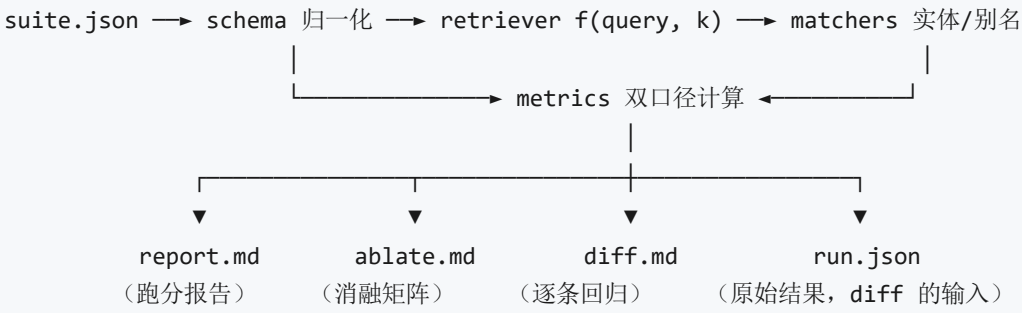
两次运行按**题目内容哈希**对齐（不是按序号，中间插入一条不会让后面全部错位），输出六类变化：

类型	含义	为什么审计需要它
`fixed`	原本未命中，现在命中	证明改动有效
`broke`	原本命中，现在未命中	总分可能仍在涨，但已经坏了东西
`improved`	排名前移但均命中	提前预警：下一条可能就掉出 Top-K
`worsened`	排名后退但均命中	同上，且真实案例中确有一条从 1 掉到 4
`added`	新增题目	题库扩张需单独可见
`removed`	题目被移除	**删掉难例同样能让总分上涨**，必须单独报出

每条退化还会附带归因：是层路由错了，还是排序掉了。这让「分数掉了 0.3」从一句抱怨变成待办清单。

三、技术架构与工程取舍

3.1 数据流



关键设计：**被测系统以纯函数形式接入**，只需满足 `f(query, k) -> list[结果]`， 结果中带 `layer` / `target` / `title`。 支持 dict、dataclass 及任何带同名属性的对象自动归一化；字段名不一致时用 `Doc.from_any(obj, field_map=...)` 指定映射。

3.2 模块划分

模块	职责
`schema`	评估集与检索结果的数据结构、类型归一化、字段映射
`matchers`	实体名匹配、别名展开、子串冲突判定（partial 识别）
`metrics`	双口径 Hit@K / MRR / 层准确率计算
`ablate`	多变体批量运行、Δ 计算、负收益告警
`diff`	内容哈希对齐、六类变化分类、单条归因
`report`	Markdown 报告渲染（并列呈现，不做取舍）
`lint`	评估集自检与泄漏画像统计
`cli`	四个子命令的入口与参数校验

3.3 三条工程原则

一、不调 LLM 打分。检索层的问题在「召回哪一条」，不在「话说得好不好」。 用 LLM 当裁判会把两个问题混在一起，而且本地小模型打分不可靠、每次运行结果还不一样。 本项目只用可判定的硬指标：期望层 + 期望目标，命中即算对。这让结果**完全可复现**， 也让它有资格去质疑别人的分数——一个自己就不稳定的工具，没有立场做审计。

二、零运行时依赖。评估工具本身要是黑盒，它的结论就没法用来判断别人。 纯标准库意味着任何人拿到代码都能逐行核对打分逻辑，也意味着它能在任何 CI 环境里跑起来， 不引入供应链风险。重依赖（向量库、embedding 模型）只出现在**被测系统一侧的适配器**里。

三、保留逐条结果。只留总分的评测撑不起回归测试。总分掉 0.3 时， 你需要知道是哪几条、坏在哪一环。因此 `run --json` 保存的原始结果是 `diff` 的直接输入，两者构成一个闭环。

四、真实场景验证（对应评审维度「场景落地」）

4.1 被测对象

对一个**已上线**的本地离线 RAG 系统 **NaviRAG v1.0.1** 做了完整审计。 该系统为游戏攻略问答场景，采用三层知识库（L1 资料 / L2 逃课 / L3 常规）+ 规则路由 + 实体硬置顶 + 别名表 + 术语增强，自报 Hit@1 94.1%。

项目	说明
知识库规模	8,428 条（ChromaDB 持久化向量库）
评估集	85 条人工编写题，8 个意图分组
数据一致性	与被测系统仓库内原文件 **SHA-256 完全一致**，未改题、未删题、未挑题
运行耗时	首次加载模型约 60 秒；85 条题跑完约 16 秒

4.2 审计结果

94.1%	92.9%	96.5%	0.951
lenient Hit@1	strict Hit@1	Hit@5	MRR

五个问题，五个答案：

审计问题	结论
94.1% 是不是口径撑大的?	**不是。**两套口径差 1.2pp，且这 1.2pp 来自**本工具严格口径的误判**（一条题命中了物品页）。真实虚高为 0
这个分数是谁挣的?	**76.5pp 来自实体名硬匹配**：关掉实体置顶后整条链路只剩 17.6%；单路纯向量仅 12.9%
真实泛化水平如何?	「描述提问」组（整句不含实体名）6 条只中 3 条，**Hit@1 为 50%**，3 条完全未命中全部落在该组
哪个模块白占位置?	术语增强 **ΔHit@1 = ±0**，MRR 仅差 0.004，在当前题集上无法证明价值
哪个部件最脆?	手工别名表值 3.5pp，关掉立刻坏 3 条，全部是俗称题；另有一条 rank 2→1、一条 rank 1→4

4.3 消融矩阵

变体	Hit@1	Δ	说明
full（线上完整链路）	94.1%	—	基准
− alias（关俗称表）	90.6%	−3.5pp	3 条俗称题失效
− enhance（关术语增强）	94.1%	±0.0pp	MRR 0.951 → 0.947，无实际收益
− entity（关实体硬置顶）	17.6%	−76.5pp	**分数的主要来源**
pure-vector（单路纯向量）	12.9%	−81.2pp	语义检索的真实水位

4.4 口径等价的证据

工具独立算出的五个数字，与被测系统自评报告**逐位吻合**：`12.9 / 17.6 / 90.6 / 94.1 / MRR 0.951`。 这说明审计口径与主流做法等价，**审计本身没有引入偏差**—— 差异全部来自本工具额外提供的信息，而不是算法分歧。这是审慎审计的前提。

> **复现边界（如实说明）。**本案例依赖被测系统本地的 ChromaDB 向量库与 embedding 模型， 外部无法直接复现跑分过程。仓库内已提供适配器源码与全部产物（原始 JSON、三份报告）供核对； **可一键复现的是 toy 演示**（10 条题，零安装，3 秒内跑完）。 我们选择把这个边界写清楚，而不是把它藏起来。

五、与同类方案的差异化

必须正面回答一个问题：这不是「把现成工具拼起来」吗？不是。三者解决的是三个不同层次的问题：

维度	RAGAudit	RAGAS	DeepEval
判定机制	确定性硬指标	LLM 裁判	LLM 裁判（G-Eval）
输入	检索结果 + 评估集	问题 / 答案 / 上下文	整条链路轨迹
输出	可信度诊断 + 归因 + 退化清单	质量分数	质量分数与断言结果
双口径量虚高	✅	❌	❌
题库泄漏画像	✅	❌	❌
删题涨分检测	✅	❌	❌
可进 CI 的秒级回归	✅（零依赖）	需 API 与网络	需 API 与网络

差异不在「指标算得更准」，而在**提问的对象不同**： RAGAS / DeepEval 问「这条答案好不好」，RAGAudit 问「你这个分数是不是真的」。 后者在前者之上，且不依赖前者。真实案例已经展示了两者的互补关系—— 被测系统答案质量并非本项目评估范围，本项目只回答分数可信度。

六、应用场景与落地路径

场景	具体用法
迭代回归（主场景）	把 `run --json` 挂进 CI，每次提交自动产出退化清单，回答「这次改动有没有让检索变差」。零依赖意味着不需要额外计费与网络。
上线前自证	用双口径 + 泄漏画像生成一份「分数可信度说明」，随技术方案一起提交，避免用一个无法追溯的百分比做承诺。
题库质量体检	对外包或自建评估集做 `lint`，先查出多少题是显式指名、多少题存在实体冲突，再决定这套题能不能用来跑分。
重构与迁移验证	换向量库、换 embedding 模型、改分块策略后，用 `diff` 逐条比对，而不是只看总分有没有掉。
第三方评审	只拿到对方的结果文件时，也能独立核算口径差异与虚高量，无需接触其内部代码。

落地门槛极低：不要求改造被测系统，不要求引入任何依赖， 只需要一个满足 `f(query, k)` 的函数。仓库内已提供一个连接真实向量库的适配器样例， 可直接作为接入模板。

七、开源治理（对应评审维度「开源治理」）

7.1 协议与依赖合规

- **项目协议**：Apache-2.0，含专利授权条款，允许商业使用与修改。
- **运行时依赖**：**零。**纯 Python 标准库实现，不存在第三方运行时依赖， 因此不存在传递依赖带来的协议风险。
- **示例依赖**：真实案例的适配器需要向量库与模型推理环境，共 **103 个传递依赖**， 但**不属于本项目运行时依赖，也不随项目分发。**
- **依赖清单**：这 103 个包由程序遍历 `importlib.metadata` **机器生成**（非手工整理）， 许可证字段按 PEP 639 优先级提取，公开在 `docs/licenses-deps.txt`。
- **合规结论**：清单中**无 GPL / AGPL**；MPL-2.0 的三个包（certifi / tqdm / orjson）属 文件级 copyleft，不影响 Apache-2.0 分发。

7.2 社区治理文件

文件	作用
`LICENSE`	Apache-2.0 全文
`CONTRIBUTING.md`	贡献流程、提交信息规范、不接受的范围（明确写着「不接受引入运行时依赖的 PR」）
`GOVERNANCE.md`	决策机制与角色定义
`CODE_OF_CONDUCT.md`	社区行为准则
`SECURITY.md`	漏洞报告渠道（GitHub 私密报告）
`ROADMAP.md`	长期规划与验收口径
`CHANGELOG.md`	按版本记录变更

7.3 质量保障

- **单元测试**：34 个用例，零依赖，**0.01 秒**跑完，覆盖双口径计算、diff 分类、报告渲染等核心路径。
- **持续集成**：GitHub Actions，每次推送自动执行全量测试。
- **可复现性**：确定性判定，无随机性、无外部服务依赖，任何机器上结果一致。
- **文档规范**：指标口径、架构、真实案例、同类对比、许可证各有独立文档， 且全部标注「已验证 / 推断」状态，不夸大结论。

7.4 第三方内容声明

真实案例中的评估集为项目作者本人手写，随被测系统以 Apache-2.0 发布； 文件中仅含题面、期望层、期望目标与分组四个字段，**不含任何游戏文本、图像或音频资源** （实测最长字段 18 个字符）。文中出现的实体名称仅用于指明评测对象，属事实性指代， 名称权利归其权利人所有。

八、长期发展规划（对应评审维度「长期发展」）

8.1 维护可行性

一个单人维护的工具，最容易死于「维护成本高于收益」。本项目在设计阶段就把这一点作为约束：

- **无依赖 → 无升级负担。**不存在「某个上游库弃坑导致项目无法安装」这一最常见的死法。
- **测试秒级 → 改动成本低。**34 个用例 0.01 秒跑完，每次改动都能立即验证。
- **边界清晰 → 需求不会失控。**明确只做检索层审计，不做 LLM 打分、不做答案质量、 不做 Agent 行为评测，避免演变成一个什么都做一点的半成品。

8.2 路线图（每项均可独立验收）

方向	内容	验收标准
框架适配器	LangChain / LlamaIndex / Chroma 结果适配层，降低接入成本	附可运行示例
报告形态	HTML 单文件报告，便于贴进 PR 与分享	无外部资源依赖
CI 集成	`--fail-on-regression N` 阈值退出码；PR 自动评论退化清单	示例工作流可跑通
题库健康度	基于 diff 的题库漂移监测，识别长期未被命中的僵尸题	能在真实题集上产出报告

8.3 风险与应对

风险	应对
单人维护，精力有限	刻意保持零依赖与小代码量（约 2,300 行含测试）；路线图只收可独立验收的小块，不做大重构
无社区支撑	以「治理意识」替代「社区规模」：贡献机制、决策规则、Issue/PR 响应预期均已书面化，任何人可据此接手
被误认为通用评测框架	文档中反复明确边界（只审检索层，不评答案质量），并与 RAGAS / DeepEval 划清分工，避免能力虚标
指标口径被质疑	所有口径定义均在 `docs/metrics.md` 中公开，且以真实案例证明与主流做法等价

九、快速验证方式

```
git clone https://github.com/MaPleooo9/RAGAudit.git
cd RAGAudit
python examples/toy/run_demo.py
```

无需 `pip install`，无需向量库与 embedding 模型，3 秒内在 `examples/toy/out/` 生成三份报告（跑分 / 消融 / 回归对比）。

命令	作用
`ragaudit lint -s suite.json`	评估集自检与泄漏画像
`ragaudit run -s suite.json -r mod:fn`	跑分并生成双口径报告
`ragaudit ablate -s suite.json -v variants.py`	消融矩阵与归因
`ragaudit diff --base a.json --cur b.json`	两次运行逐条对比

十、附录：文档索引

文档	内容
`README.md`	项目定位、四个功能、接入方式、设计取舍
`docs/metrics.md`	每个指标的计算方式与设计理由
`docs/architecture.md`	模块划分与数据流
`docs/case-navirag.md`	真实案例：审计一个 94.1% 的系统
`docs/comparison.md`	与 RAGAS / DeepEval 的完整对照
`docs/licenses.md`	依赖许可证清单与合规结论
`ROADMAP.md`	长期规划